MODULE 2

WEBSOCKET COMMUNICATION WITH A LOCAL SERVER

OBJECTIVES

1) Understand the HTTP (Hypertext Transfer Protocol) protocol.
2) Understand HTTP communication.
3) Understand the use of WebSocket in an IoT-system context.

REQUIREMENTS

1) Arduino IDE/PlatformIO.
2) ESP32/ESP8266.
3) Local server (Raspberry Pi 3B).
4) Sensor (opsional).
5) Aktuator (opsional).

2.1 Hypertext Transfer Protocol (HTTP)

HTTP (Hypertext Transfer Protocol) is an application-layer protocol for exchanging resource representations between clients and servers. A client sends a request containing a method, target, headers, and, when required, a body. The server then sends a response containing a status, headers, and a data representation. HTTP can be used for documents, structured data, images, video, and API services over either a local network or the internet. A web browser is one type of HTTP client, but HTTP is also used by mobile applications, IoT devices, and inter-system services.

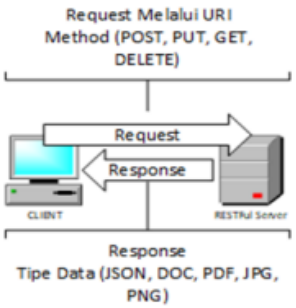


Figure 2.1: HTTP visualization

2.1.1 CRUD

CRUD stands for Create, Read, Update, and Delete. The term describes four basic data operations and can be applied to various storage systems, not only relational databases. In HTTP-based systems, CRUD operations are often mapped to HTTP methods, although the semantics of HTTP methods remain defined by the protocol specification and do not always correspond exactly to database operations.

The CRUD functions are as follows.

- **CREATE:** Creates a new entry or record in a database table. In SQL-based applications, this operation is commonly performed with `INSERT`.
- **READ:** Retrieves a data entry stored in a database without modifying the stored data.
- **UPDATE:** Modifies an existing data entry when its information must be changed.
- **DELETE:** Removes a data entry that is no longer required from a database.

Several naming variants have been developed from the CRUD concept. The following terms are not part of the HTTP standard; they are application-operation naming patterns:

- BREAD (Browse, Read, Edit, Add, Delete)
- CRAP (Create, Replicate, Append, and Process)
- CRUDL (Create, Read, Update, Delete, List)
- DAVE (Delete, Add View, Edit)

2.1.2 HTTP Communication Methods

HTTP methods have their own semantics and can be used to support CRUD operations in an API:

- **GET** requests a representation of a resource and should not change server state.
- **POST** sends data to be processed according to the resource function; it is often used to create a new entry.
- **PUT** creates or replaces the representation of a resource at the specified target.
- **PATCH** applies a partial modification to a resource.
- **DELETE** requests removal of the resource association at the specified target.

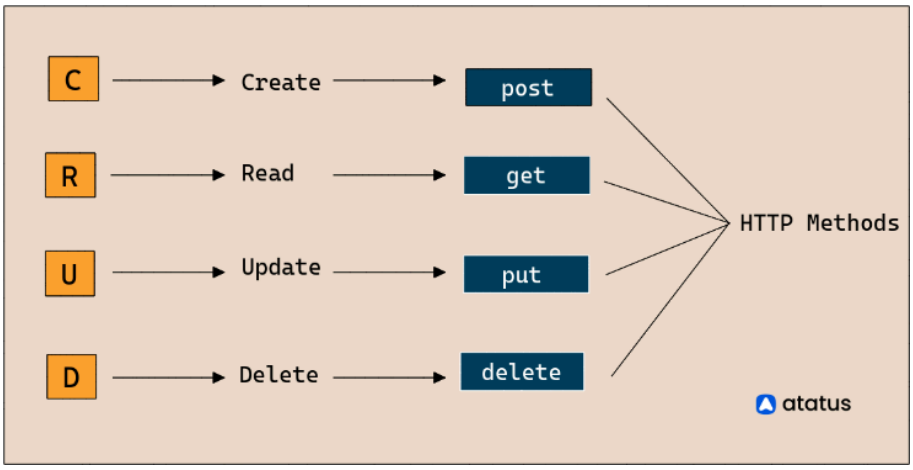


Figure 2.2: HTTP communication methods

In addition to the methods described above, HTTP provides several other communication methods, as shown in the following table.

Table 2.1: Types and functions of HTTP methods

HTTP Method	Definition	Explanation
GET	Retrieve a resource	Used to retrieve specified data from a server
POST	Process data	Used to send data for processing according to the resource function
PUT	Create or replace a resource	Used to create or replace a resource representation at the target
DELETE	Delete a resource	Used to remove specified data from a server
PATCH	Modify part of a resource	Used to update or modify part of existing data on a server
HEAD	Obtain header information	Used to request the headers for data that would be returned by GET
OPTIONS	Obtain server capability information	Used to request information about the methods supported by the server for a target resource
CONNECT	Establish a tunnel	Used to request that a proxy establish a tunnel to the destination server
TRACE	Perform diagnostics	Used to request that the final recipient return the request it received

2.2 WebSocket

WebSocket is a protocol that provides bidirectional communication over a single TCP connection. The connection begins with an HTTP Upgrade handshake, after which the client and server can exchange frames without creating a new HTTP request for every message. WebSocket is suitable for applications that require interactive or high-frequency data exchange, although actual latency is still affected by the network, implementation, and server load.

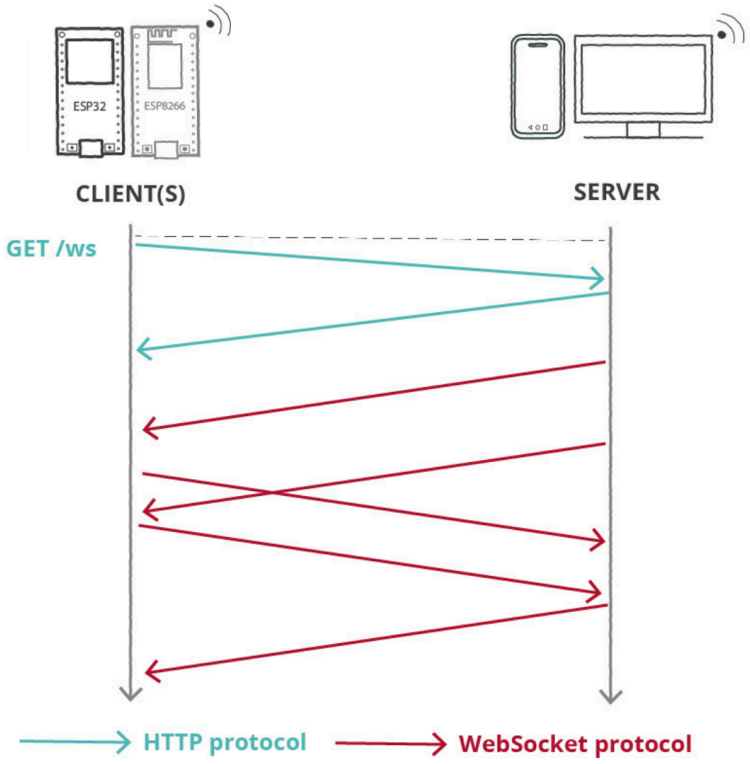


Figure 2.3: Websocket Communication Flow

The WebSocket communication flow is as follows:

a. The client sends a special HTTP request asking to upgrade the connection to WebSocket. The following is an example request header:

```
GET /chat HTTP/1.1
Host: example.com
Upgrade: WebSocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
Origin: https://example.com
```

When the server accepts the request, it responds with:

```
HTTP/1.1 101 Switching Protocols
Upgrade: WebSocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
```

b. After the handshake succeeds, the client and server can exchange frames. A WebSocket frame still has a protocol header, but it does not carry the complete HTTP headers with every message. The frame structure contains the following elements:

- **FIN bit:** Indicates whether the frame is the final frame of a message.
- **Opcode:** Identifies the frame type (text, binary, close, ping, or pong).
- **Mask bit:** Indicates whether the payload is masked. Client-to-server frames must be masked, while server-to-client frames are not masked. Masking is not encryption and does not provide confidentiality.
- **Payload length:** The size of the transmitted message data.
- **Payload data:** The transmitted data content.

The following table compares WebSocket communication with traditional HTTP communication:

Table 2.2: Differences between WebSocket and traditional HTTP

Aspect	HTTP	WebSocket
Communication Pattern	Uses a request-response pattern; the client generally initiates the exchange	Maintains the connection after the handshake so that the client and server can both send messages
Connection Overhead	Every request and response carries HTTP headers; their size depends on the configuration	After the handshake, frame overhead varies with payload length, masking, and extensions
Latency	Requires a request-response exchange for each interaction	Reduces the need for a new handshake or request for every message, but does not eliminate network latency
Scalability	Its stateless nature generally makes horizontal scaling easier	WebSocket connections are stateful, making scaling more complex

2.3 Implementing the WebSocket Protocol

2.3.1 Raspberry Pi Python Server

Before communication can take place, a WebSocket server must be created so that the ESP32 can connect and exchange messages. Create a new folder to store the server code, for example `server-websocket`. Install the required Python packages if they are not already installed:

Installing Python Packages and a Virtual Environment

```
sudo apt update
sudo apt install python3 python3-pip python3-venv
```

Next, create a virtual environment inside the new folder as follows:

Creating a Python Virtual Environment

```
python3 -m venv .venv

# Activate the virtual environment
source .venv/bin/activate
```

After activating the virtual environment, install the `websockets` package for Python by running the following command in the terminal:

Installing the Python WebSocket Package

```
pip install websockets
```

Create a Python source file in the folder, for example `main.py`, and enter the following content:

Importing Libraries and Configuring Variables

```
import asyncio  # Supports asynchronous coroutine execution

# WebSocket library
from websockets.asyncio.server import ServerConnection, serve
from websockets.exceptions import ConnectionClosed

HOST: str = "0.0.0.0"  # Listen on all network interfaces
PORT: int = 8765
```

Callback Function for Multiple Clients

```
async def handle_client(websocket: ServerConnection) -> None:
    # Distinguish messages by the WebSocket path
    # The path used in this example is "/esp32"
    path: str = websocket.request.path

    # Select the path to process
    if path == "/esp32":
        print("ESP32 device connected")
        try:
            async for message in websocket:
                # Message from ESP32
                print( f"ESP32 message: {message}" )

                # Reply
                response: str = ( f"ACK from Raspberry Pi: {message}" )
                await websocket.send(response)

        except ConnectionClosed:
            print("ESP32 disconnected")
    # Add other paths as required
    # Default case for an unrecognized path
    else:
        print(f"Unknown path: {path}")
```

In the code above, the `async` keyword defines a coroutine that can run asynchronously. The `websocket` object represents one client connection. Messages are received through the `async for` loop, while `websocket.send()` sends a response over that connection.

Python Main Function

```
async def main() -> None:
    # Use the configuration defined above
    async with serve(handle_client, HOST, PORT):
        print(f"WebSocket server running " f"on {HOST}:{PORT}")
        # Wait for messages...
        await asyncio.Future()

if __name__ == "__main__":
    asyncio.run(main())
```

The `await` keyword suspends the coroutine while it waits for an asynchronous operation without blocking the entire event loop. This mechanism allows the server to handle multiple connections on the Raspberry Pi's IP address and port.

After creating the source file, run the Python server `main.py` from the terminal.

Running the Python Server

```
python main.py
```

2.3.2 ESP32 Source Code

Before writing the ESP32 code, obtain the IP address of the Raspberry Pi configured as the server. Run the following command in the terminal to display the Raspberry Pi's IP address:

Viewing the Raspberry Pi IP Address

```
hostname -I
# Alternatively, use the following command
ip addr
```

The commands above display the Raspberry Pi's IPv4 and IPv6 addresses. Only the IPv4 address is required for this WebSocket server, for example `192.168.1.100`.

Information
This practicum uses the `WebSockets` library by Markus Sattler:
https://github.com/Links2004/arduinoWebSockets

Install the required library first. Then import it into the ESP32 program as usual.

Libraries Used on the ESP32

```
#include <WiFi.h>             // Wi-Fi connection
#include <WebSocketsClient.h> // WebSocket connection
```

Wi-Fi and WebSocket Configuration

```
// WiFi
const char* WIFI_SSID     = "YOUR_WIFI_NAME";
const char* WIFI_PASSWORD = "YOUR_WIFI_PASSWORD";

// WebSocket
const char*    WS_HOST = "192.168.1.100";  // Match the Raspberry Pi IP address
const uint16_t WS_PORT = 8765;             // Match the Raspberry Pi port

// Use different paths to distinguish data or endpoint types
const char*    WS_PATH = "/esp32";

WebSocketsClient webSocket;

bool webSocketConnected = false;
unsigned long previousSendTime = 0;
const unsigned long SEND_INTERVAL = 1000; // Data-transmission interval

uint32_t counter = 0;
```

Ensure that the IP address and port in the ESP32 code match those configured on the Raspberry Pi.

WebSocket Callback Function

```
void webSocketEvent(WStype_t type, uint8_t* payload, size_t length) {
    switch (type) {
        case WStype_CONNECTED:
            webSocketConnected = true;

            Serial.println("WebSocket connected.");

            // Optional initial message
            webSocket.sendTXT("{\"status\":\"ESP32-S3 connected\"}");
            break;

        case WStype_DISCONNECTED:
            webSocketConnected = false;
            Serial.println("WebSocket disconnected.");
            break;

        case WStype_TEXT:
            Serial.print("Message from server: ");
            Serial.write(payload, length);
            Serial.println();
            break;

        case WStype_ERROR:
            Serial.println("WebSocket error.");
            break;
```

```
26
27          default:
28              break;
29      }
30  }
```

In the code above, the `void webSocketEvent()` callback processes WebSocket events for both transmitted and received messages.

Connecting to Wi-Fi

```
1   void connectToWiFi() {
2       Serial.print("Connecting to Wi-Fi");
3
4       WiFi.mode(WIFI_STA);
5       WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
6
7       // Wait for the Wi-Fi connection
8       while (WiFi.status() != WL_CONNECTED) {
9           delay(500);
10          Serial.print(".");
11      }
12
13      Serial.println();
14      Serial.println("Wi-Fi connected.");
15
16      Serial.print("ESP32 IP address: ");
17      Serial.println(WiFi.localIP());
18  }
```

Before using WebSocket, connect the ESP32 to the Wi-Fi network. The `WIFI_STA` mode allows the device to operate as a client that connects to an available Wi-Fi access point (AP), similarly to a mobile phone connecting to a Wi-Fi router. The connection is initiated by calling `WiFi.begin()` with the Wi-Fi name or SSID and password. Ensure that the ESP32 and Raspberry Pi are connected to the same local network.

ESP32 Setup Function

```
1   void setup() {
2       Serial.begin(115200);
3       delay(1000);
4
5       connectToWiFi();
6
7       webSocket.begin(WS_HOST, WS_PORT, WS_PATH);  // Start the WebSocket connection
8       webSocket.onEvent(webSocketEvent);    // Register the callback function
9       webSocket.setReconnectInterval(5000); // Attempt reconnection every 5 seconds
10
11      // Reconnect to Wi-Fi automatically after disconnection
12      WiFi.setAutoReconnect(true);
13  }
```

WebSocket starts after the ESP32 successfully connects to the network. In the code above, the ESP32 attempts to connect to the Raspberry Pi through WebSocket using the configured host, port, and path. If the connection becomes unstable, Wi-Fi and WebSocket attempt to reconnect, with a five-second WebSocket reconnection interval.

ESP32 Loop Function

```
1   void loop() {
2       // Process the WebSocket connection
3       webSocket.loop();
```

```
4
5       unsigned long currentTime = millis();
6
7       if (
8           webSocketConnected &&
9           currentTime - previousSendTime >= SEND_INTERVAL
10      ) {
11          previousSendTime = currentTime;
12
13          // Place the transmission logic in the following block
14          // This example sends a counter value every second
15          String message = "{\"counter\":";
16          message += counter;
17          message += "}";
18
19          webSocket.sendTXT(message);
20
21          Serial.print("Sent: ");
22          Serial.println(message);
23
24          counter++;
25      }
26  }
```

The `webSocket.loop()` method must be called regularly to process WebSocket connection events. After the transmission interval has elapsed, `webSocket.sendTXT()` sends a text message to the Raspberry Pi server.

```
ESP32 device connected
ESP32 message: {"status":"ESP32-S3 connected"}
ESP32 message: {"counter":0}
ESP32 message: {"counter":1}
ESP32 message: {"counter":2}
ESP32 message: {"counter":3}
ESP32 message: {"counter":4}
ESP32 message: {"counter":5}
```

Figure 2.4: WebSocket server output

The example above shows the output generated by the Python WebSocket server created earlier.

APPENDIX: MODULE 2

WebSocket Python Server Source Code

WebSocket Python Server

```python
import asyncio # Supports asynchronous coroutine execution

# WebSocket library
from websockets.asyncio.server import ServerConnection, serve
from websockets.exceptions import ConnectionClosed

HOST: str = "0.0.0.0" # Listen on all network interfaces
PORT: int = 8765

async def handle_client(websocket: ServerConnection) -> None:
  # Distinguish messages by the WebSocket path
  # The path used in this example is "/esp32"
  path: str = websocket.request.path

  # Select the path to process
  if path == "/esp32":
    print("ESP32 device connected")

    try:
      async for message in websocket:
        # Message from ESP32
        print( f"ESP32 message: {message}" )

        # Reply
        response: str = ( f"ACK from Raspberry Pi: {message}" )
        await websocket.send(response)
    except ConnectionClosed:
      print("ESP32 disconnected")

  # Add other paths as required
  # Default case for an unrecognized path
  else:
  print(f"Unknown path: {path}")

async def main() -> None:
  # Use the configuration defined above
  async with serve(handle_client, HOST, PORT):
  print(f"WebSocket server running " f"on {HOST}:{PORT}")

  # Wait for messages...
  await asyncio.Future()

if __name__ == "__main__":
  asyncio.run(main())
```

WebSocket Source Code for ESP32

WebSocket with ESP32

```cpp
#include <WiFi.h>
#include <WebSocketsClient.h>

const char* WIFI_SSID     = "YOUR_WIFI_NAME";
const char* WIFI_PASSWORD = "YOUR_WIFI_PASSWORD";

const char*    WS_HOST = "192.168.1.100";
const uint16_t WS_PORT = 8765;
const char*    WS_PATH = "/esp32";

WebSocketsClient webSocket;

```

```
bool webSocketConnected = false;
unsigned long previousSendTime = 0;
const unsigned long SEND_INTERVAL = 1000;
uint32_t counter = 0;

void webSocketEvent(WStype_t type, uint8_t* payload, size_t length) {
  switch (type) {
    case WStype_CONNECTED:
      webSocketConnected = true;
      Serial.println("WebSocket connected.");
      webSocket.sendTXT("{\"status\":\"ESP32 connected\"}");
      break;

    case WStype_DISCONNECTED:
      webSocketConnected = false;
      Serial.println("WebSocket disconnected.");
      break;

    case WStype_TEXT:
      Serial.print("Message from server: ");
      Serial.write(payload, length);
      Serial.println();
      break;

    case WStype_ERROR:
      Serial.println("WebSocket error.");
      break;

    default:
      break;
  }
}

void connectToWiFi() {
  Serial.print("Connecting to Wi-Fi");
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println();
  Serial.println("Wi-Fi connected.");
  Serial.print("ESP32 IP address: ");
  Serial.println(WiFi.localIP());
}

void setup() {
  Serial.begin(115200);
  delay(1000);

  connectToWiFi();

  webSocket.begin(WS_HOST, WS_PORT, WS_PATH);
  webSocket.onEvent(webSocketEvent);
  webSocket.setReconnectInterval(5000);
  WiFi.setAutoReconnect(true);
}

void loop() {
  webSocket.loop();

  unsigned long currentTime = millis();
  if (
    webSocketConnected &&
```

```
80        currentTime - previousSendTime >= SEND_INTERVAL
81      ) {
82        previousSendTime = currentTime;
83
84        String message = "{\"counter\":";
85        message += counter;
86        message += "}";
87
88        webSocket.sendTXT(message);
89        Serial.print("Sent: ");
90        Serial.println(message);
91        counter++;
92      }
93    }
```